Instructor Handout 1: Computer Architecture, Organization, and Performance

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Instructor Copy — Not for Student Distribution

1.1 Before You Teach This Lecture

- **Patterson & Hennessy, *Computer Organization and Design*, ARM Edition** — Sec. 1.1 "Computer Abstractions and Technology," p.3 (architecture vs. organization, classes of computers); Sec. 1.4 "Under the Covers," p.16 (basic structure); Sec. 1.6 "Performance," p.28 (performance equation, CPI, MIPS, Amdahl's Law). Page-cited — index built 2026-08-07. This is your precise, page-citable anchor for the quantitative core.
- **Hamacher, *Computer Organization*, Ch. 1** — *not yet page-indexed.* Cite at chapter level; the source for classes of computers and the general framing.
- **Plan the two worked machines before class** (the running thread): program P = 10^7 instructions; Machine A (2 GHz, CPI 2) vs. Machine B (3 GHz, CPI 3), then B's CPI dropped to 1.5. Do the arithmetic yourself first so the numbers on the board are right the first time.
- **Decide your benchmark story.** The performance/measurement section needs one concrete "real machine" framing — a processor model and clock rate you can name. P&H's Sec. 1.6 uses the Core i7; if you want a local, simpler anchor, pick any modern x86/ARM part you can state specs for.

1.2 Architecture vs. Organization, and Basic Structure

Open the course by stating the big question this course answers: a program is written in a high-level language, compiled to machine instructions, and run on hardware — what sits *between* the program and the silicon? Answer it in two words: the machine. Then define the two terms that frame the entire course, and have students say which is which before you confirm.

Definition (Computer Architecture). *The programmer-visible interface of a machine — its instruction set, registers, data types, and addressing modes. Two machines with the same architecture run the same programs.*

Definition (Computer Organization). *The hardware implementation of that interface — the datapath, control unit, buses, and memory hierarchy. Same architecture, different organization, different speed.*

State the punchline plainly: same architecture ⇒ same programs run; different organization ⇒ they run at different speeds and costs. P&H frame it as the "what" (the programmer's contract) versus the "how" (the hardware's construction) — Sec. 1.1, p.3 [1].

> **Teaching note:** The student trap in this section is treating architecture and organization as the *same thing* or as a mere vocabulary pair. The teaching move is to make the distinction do work immediately: "why has x86 survived for decades even though the hardware underneath it is completely different now?" The answer (stable architecture, changing organization) makes the distinction a *reason*, not a label.

Cover the classes of computers as a class discussion (personal, server, supercomputer, embedded, personal mobile) — ask students which computers they own and what each one optimizes. Extract the single lesson: the design goal differs by class, so "best" is not a universal adjective (Hamacher, Ch. 1; P&H, Sec. 1.1, p.3).

Then draw the basic structure (Figure 1.1).

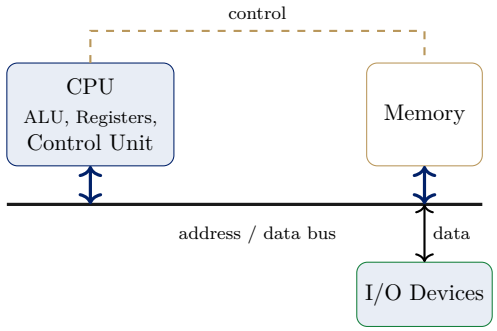

Figure 1.1: The basic structure of a computer — cf. P&H, Sec. 1.4, p.16: CPU (ALU + registers + control) connected to memory and I/O by buses [1].

How to present it: draw the three boxes first (CPU left, memory top-right, I/O bottom-right), then the horizontal bus between CPU and memory labeled *address / data*, then the vertical drop to I/O. Add the dashed control line last and label it *control* — narrating that control travels separately from data. The separation of "address/data" from "control" is the detail students most often miss and is worth a beat.

> **Teaching note:** Students will want the CPU box to be a single block. Pause on the internal contents (ALU, registers, control unit) and note: the next six weeks are about what is *inside* that box. This lecture is the only one where the CPU is a black box — later lectures open it up.

Close by previewing the instruction cycle at one sentence's depth: the CPU repeatedly fetches, decodes, executes. Today we need only the consequence — that execution time is built from instruction count (IC), cycles per instruction (CPI), and clock rate (f). Week 4 dissects the cycle itself.

1.3 The Basic Performance Equation

Pose the Socratic question that opens the section: two machines run the same program; one has a faster clock, the other executes fewer instructions. Which is faster? Let students argue, then land the principle: the only honest measure is **time**; everything else is a component of time.

Definition (Basic Performance Equation).

$$CPU\ time = IC \times CPI \times T_c = \frac{IC \times CPI}{f},$$

where IC is the instruction count, CPI the average cycles per instruction, T_c the clock cycle time, and $f = 1/T_c$ the clock rate.

Introduce the three factors with who-controls-what (Table 1). The table is the "so what" of the whole equation: each factor is a separate design lever, owned by a different part of the design, and improving one can hurt another.

Factor	What it measures	Influenced by	Changed by
IC	instructions per program	ISA, compiler, program	architecture
CPI	cycles per instruction	datapath, pipeline, memory	organization
f (T_c)	clock rate	fabrication, circuit design	technology + design

Table 1: The three factors of the performance equation and what sets each one.

> **Teaching note:** The most common error is treating a "faster clock" as automatically "a faster machine." The worked tie below exists precisely to kill that intuition. If you can only teach one example this lecture, teach the tie — it is the example that makes the equation necessary rather than decorative.

Example 1.3.1 (Two machines, one program — a tie)**.** *Machine A: 2 GHz, CPI 2; Machine B: 3 GHz, CPI 3; program $P = 10^7$ instructions.*

- `A:` *$time = 10^7 \times 2 \times 0.5ns = 10$ ms.*
- `B:` *$time = 10^7 \times 3 \times \frac{1}{3}ns = 10$ ms.*

Tie, despite B's faster clock — B's higher CPI cancels it. Faster clock $\not\Rightarrow$ faster machine.

How to present it: write the equation, then fill in A and B *side by side on the board*, one line at a time, so the class watches the tie emerge rather than hearing "they tie." The punchline — "faster clock $\not\Rightarrow$ faster machine, the equation decides" — lands only if they saw the numbers arrive at the same answer.

Example 1.3.2 (Breaking the tie by improving CPI)**.** *B's designer reduces CPI from 3 to 1.5 (better pipelining): $time = 10^7 \times 1.5 \times \frac{1}{3}ns = 5$ ms — now B is 2× faster, via CPI (organization), not clock rate.*

A question worth asking live, after the second example: "where did that 2× come from — the clock, the ISA, or the organization?" The answer (organization, via CPI) is the forward hook to Weeks 5–7, where the course builds the datapath and control that set CPI.

> **Teaching note:** Resist the urge to skip the tie example because "the numbers are round and obvious." The tie is not a trivial coincidence — it is the counterexample that breaks the intuitive (wrong) belief that clock rate equals speed. Spend the time; it pays for itself for the rest of the course.

1.4 Measuring and Comparing Performance

State the measurement principle first: the right measure is **execution time** on a representative workload (or its inverse, throughput). Distinguish **wall-clock time** (everything counts) from **CPU time** (only the CPU's computing time) — the latter isolates processor design.

Then introduce MIPS with its caveat.

Definition (MIPS).

$$MIPS = \frac{IC}{execution\ time \times 10^6} = \frac{f}{CPI \times 10^6},$$

Million Instructions Per Second. Misleading when instruction sets differ.

Give the three reasons MIPS misleads (P&H, Sec. 1.6, p.28) [1]: it ignores what each instruction does; it varies with the program; two machines can report the same MIPS yet have different execution times.

Example 1.4.1 (The MIPS trap when instruction sets match)**.** *Machine A: 1 GHz, CPI 2.0, IC* $= 10^6$*. Machine B: 1 GHz, CPI 1.0, same P.*

- `A:` $MIPS = 500$*, time* $= 2$ *ms.*
- `B:` $MIPS = 1000$*, time* $= 1$ *ms.*

Here MIPS and time agree (2×), because the instruction sets match. The trap appears only when instruction sets differ.

> **Teaching note:** Students love MIPS because it is one number that sounds like speed. The teaching move is to let them use it, then produce the counterexample where it fails — two machines, same reported MIPS, very different time because one instruction does more work. The rule that sticks: "report time; use MIPS with a caveat." State the rule, don't just imply it.

Close the measurement section with the Socratic check: Machine C runs 500M instructions in 1 s; Machine D runs 800M in 1.6 s. Which is faster? (C — 1 s vs. 1.6 s, despite fewer instructions.) The point: instruction count is a component, not a speed metric.

1.5 Amdahl's Law

Motivate with the puzzle: a program takes 100 s; you make multiplies 10× faster, but multiplies are 20% of the time. Ask the class to guess the whole-program speedup before revealing 1.22×. Let the guesses (including "10×") land, then show the arithmetic: $80 + 2 = 82$ s.

Definition (Amdahl's Law). *If a fraction* f *of execution time can be sped up by a factor* S_f*, the overall speedup is*

$$Speedup = \frac{1}{(1-f) + \frac{f}{S_f}}.$$

As $S_f \to \infty$*, speedup* $\to 1/(1-f)$*: the unimproved fraction sets a hard ceiling.*

How to present it: draw the two axes, then the dashed ceiling line at $1/(1-f)$ *before* the curve, and label it "the ceiling." Then draw the curve rising steeply and flattening. Reading the saturation off the picture — "adding more to S_f gains almost nothing" — is the whole law in one look.

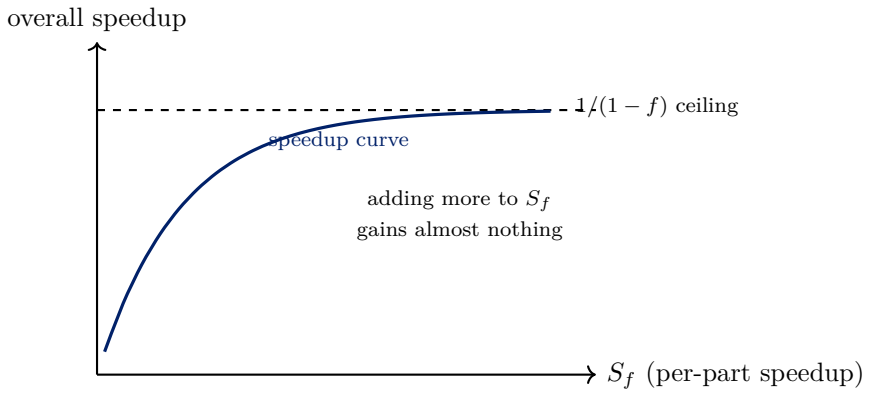

Figure 1.2: Amdahl's Law: overall speedup saturates at $1/(1-f)$ — an infinite per-part speedup cannot beat the unimproved fraction.

> **Teaching note:** The single most important move in this section is to make students compute the ceiling *before* the general formula: $1/(1-f)$ as "the part you did not improve is the part you cannot fix." Once they feel why the 60% (or 70%) unimproved fraction dominates, the formula is just notation for what they already derived.

Example 1.5.1 (Applying Amdahl's Law)**.** *40% FP, new FP unit 4× faster: Speedup* $= 1/(0.6 + 0.1) = 1/0.7 \approx 1.43\times$.

Example 1.5.2 (The ceiling in action)**.** *40% FP, FP unit infinitely fast:* $\lim = 1/0.6 \approx 1.67\times$. *To reach 3× overall, the FP unit is the wrong investment — the other 60% must also improve.*

> **Teaching note:** The design-implication reading of Amdahl's Law — "where not to invest" — is what separates students who can compute it from students who can *use* it. Ask the design question live: "if a program must run 3× faster and FP is 40%, where do you invest?" The answer (also the other 60%) is the point of the whole section.

A question worth asking live, near the end: "is the ceiling real, or is it a pessimistic assumption?" The twist (Section 1.5.1 below) is worth stating even if time is short — it inoculates students against over-quoting Amdahl.

1.5.1 The Parallelism Twist

Amdahl's Law is often quoted against parallel computing: "even with infinite processors, the serial fraction limits speedup." The statement is right for fixed-size workloads, but it assumes f is **fixed**. If the problem *grows* (larger input), the parallel fraction usually grows with it, so larger problems extract more parallelism and soften the ceiling. Amdahl's Law is about fixed-size workloads, not scaling problems.

> **Teaching note:** This is a one-minute "so you know the caveat" moment, not a second lesson. State it, then move on — Week 11 (parallelism) will return to it properly. The goal here is only to stop students from confidently quoting Amdahl as a universal truth.

1.6 Synthesis and Summary

Close the lecture by closing the thread: the running comparison — A (2 GHz, CPI 2) and B (3 GHz, CPI 3) tied at 10 ms; improving B's CPI to 1.5 made it 2× faster — was a CPI (organization) win, not a clock-rate win. The thread used every tool of the chapter: the equation to compute time, the factor table to see which lever was pulled, and time (not MIPS) as the metric.

Set up Week 2's hook: the performance equation assumes we can count instructions and CPI, but before the CPU can execute anything, the data must be represented in bits and arithmetic implemented in hardware. Week 2: number systems (two's complement) and adders/subtractors.

Summary — quick reference: *Architecture* = programmer's interface; *organization* = hardware implementation; same architecture, different organization, different speed. *Basic structure:* CPU (ALU + registers + control) $\leftrightarrow$ memory $\leftrightarrow$ I/O via buses. *Performance equation:* CPU time = IC $\times$ CPI/f, three independent factors. *Measurement:* time (or throughput) on a representative benchmark; beware MIPS. *Amdahl's Law:* Speedup = $1/[(1-f)+f/S_f]$; the unimproved fraction sets the ceiling.

References

[1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.